Student Engineering Reflection: Cloud Knowledge Distillation & Deployment

Student Name: Nemapu Navaneeth	**Roll Number:** 230101086
Course: CS3081 Cloud Computing	**GitHub:** https://github.com/NavaneethNemapu/cloud-command-parser

1. Project Overview & Motivation

When I started this project, the core engineering challenge was clear: modern multi-billion parameter language models cannot be deployed realistically on low-end edge devices or budget laptop environments due to severe CPU and RAM bottlenecks. In this assignment, I tackled this problem by using sequence-level knowledge distillation (Kim & Rush, 2016) to compress a 3.8B parameter teacher model (Phi-3.5-mini-instruct) into an ultra-compact 70M parameter causal student model (EleutherAI/pythia-70m). I then built a production-style cloud deployment pipeline: partitioning and cleaning training data, storing artifacts in an S3-compatible object store (SeaweedFS), containerizing the service with strict resource limits (1 vCPU, 1 GiB RAM), deploying it to a local k3d Kubernetes cluster, configuring autoscaling and health probes, designing stateless network security rules (NACLs), and setting up automated CI/CD quality gates.

2. What I Learnt (Deep Technical Learnings)

A. Sequence-Level Knowledge Distillation in Practice:

I previously understood distillation only conceptually. Implementing it taught me the practical distinction between token-level and sequence-level distillation. Token-level distillation requires storing and matching soft probability logits across the entire vocabulary (50,000+ floats per token), which is computationally prohibitive on CPU and impossible when teacher and student share different tokenizers. In contrast, sequence-level distillation treats the teacher's greedily decoded output strings as ground-truth targets. The student learns to mimic the teacher's structured behavior through standard cross-entropy loss over text sequences, transferring domain knowledge efficiently without needing teacher logits or GPU infrastructure.

B. PyTorch Autograd Engine & CPU Hardware Execution Constraints:

Training on a CPU was an eye-opening lesson in hardware-software interaction. On GPUs, high tensor parallelism masks output projection costs. On CPU, however, backpropagation through the final unembedding layer (`embed_out`) becomes a massive memory-bandwidth bottleneck. Multiplying large vocabulary matrices across unpadded token sequences saturated the CPU cache. I learned that optimizing intra-op threads (`torch.set_num_threads`), enforcing dynamic batch padding, and keeping sequence lengths compact are mandatory techniques for feasible CPU deep learning.

C. S3 Object Storage & Distributed Coordination:

Setting up SeaweedFS showed me how cloud-native object stores decouple metadata management (LevelDB on the Filer) from raw binary chunks (Volume Servers). Building independent worker processes across object storage taught me how distributed concurrency must be handled: I implemented atomic directory locking (`os.mkdir`) as a lightweight filesystem mutex to eliminate race conditions during concurrent queries to Ollama without deadlocking.

D. Kubernetes Lifecycle Probes & Stateless Networking:

Configuring Kubernetes pods gave me hands-on appreciation for why readiness probes (`/readyz`) are separated from liveness probes (`/healthz`). While a pod is alive, it must not receive user traffic until its model weights are fully downloaded from S3 and loaded into RAM. Additionally, working with subnetting and stateless NACLs taught me that network rules must explicitly account for the return path: outgoing HTTP requests work only if ephemeral port ranges (1024–65535) are explicitly permitted for incoming replies.

3. Major Engineering Challenges & Implemented Solutions

Challenge 1: The 15-Minute CPU Training Step Bottleneck in PyTorch

What Happened: When I initiated fine-tuning with an initial batch size of 50 and sequence length of 48, a single batch execution stalled on CPU for over 15 minutes, making 3 epochs of training impossible.
Diagnostic: I profiled the PyTorch autograd engine to identify where cycles were spent. Profiling revealed that **99.48% of CPU backward execution time** was consumed by `aten::mm`, with **79.19% concentrated in MmBackward0** (the gradient of the output projection layer `embed_out`). Because Pythia-70M has a vocabulary of 50,304 tokens and a hidden size of 512, computing gradients for 2,400 active tokens per batch produced ~250 GFLOPs of dense computation that choked the CPU memory bus.
Solution: I redesigned the pipeline: I dropped `batch_size` to 4, capped `max_seq_length` to 48 with dynamic collator padding, and tuned intra-op threads to 4. This reduced active tokens per step to 192 (a 12.5x drop) and slashed step execution time to ~28.9 seconds.

Challenge 2: SeaweedFS Master Volume Saturation Across S3 Buckets

What Happened: When uploading the trained model artifacts to the `models` S3 bucket, SeaweedFS rejected writes with internal RPC errors: `'failed to find writable volumes for collection: models: No writable volumes'`.
Diagnostic: Inspecting the cluster via `weed shell volume.list` showed that SeaweedFS had defaulted to `volume.max=8`. All 8 volume slots had been claimed by the default and dataset collections, leaving zero free slots to allocate for the new `models` collection.
Solution: I modified the SeaweedFS startup parameters by passing `"-volume.max=50"` and `"-s3.port=8333"`, and cleared empty volumes with `volume.deleteEmpty`. This created 42 free volume slots, resolving the volume exhaustion issue immediately.

Challenge 3: Stateless NACL Packet Dropping on Ephemeral Return Ports

What Happened: When testing network access control rules against test packets with `nacl_sim.py`, valid client packets and backend responses were dropped unexpectedly.
Diagnostic: Stateless NACLs do not have a connection state tracking table. Unlike stateful firewalls, every packet is evaluated in isolation. When an external client makes an HTTPS call on port 443, the server's reply is sent back to the client's randomly assigned ephemeral port (1024–65535). Without an explicit outbound rule permitting return traffic to ephemeral ports, the packet gets blocked.
Solution: I mapped all communication paths symmetrically in `network/nacl.json`: allowing outbound ephemeral ports for client responses, and inbound ephemeral ports for backend app responses. In addition, I placed the block rule for the abusive subnet `198.51.100.0/24` at Rule 100 so it evaluates first before any broader allow rules, achieving 100% test pass rate.

Challenge 4: Gradient Explosion & Divergence on Small-Batch CPU Training

What Happened: During initial fine-tuning trials with a learning rate of 5e-4, the loss abruptly exploded to NaN on batch 2.
Diagnostic: Small batch sizes introduce high gradient variance in transformer attention layers, causing parameter updates to overshoot.
Solution: I tuned the learning rate down to `0.0001` (1e-4) and added gradient norm clipping (`clip_grad_norm_(1.0)`). This completely stabilized training, resulting in smooth loss convergence from 3.6991 (Epoch 1) to 1.5500 (Epoch 3).

4. Key Empirical Findings & Measured Observations

• **Compression vs. Quality Retention:** Distilling Phi-3.5 into Pythia-70M produced a **97.7% parameter reduction** (from 3.8B to 70M). Despite this massive compression, the distilled student achieved **97.0% JSON validity** and **59.0% full-frame accuracy** on the 100-sample validation set (well exceeding the required threshold of 31.0%, which is half of the teacher's 62.0% full-frame accuracy). The untrained model scored 0.0%, demonstrating that sequence-level distillation successfully transferred domain parsing rules.
• **Memory Stability Under Hard Limits:** Over 50 sequential requests under 1 vCPU and 1 GiB limits, baseline memory with weights loaded was 458 MiB, and peak memory reached **554.00 MiB**. The service maintained **470 MiB (45.9%) of safe headroom** below the 1,024 MiB limit with 0 out-of-memory crashes.
• **Cold-Start Latency Breakdown:** Measuring pod cold start over 3 runs gave: Run 1: 10.75s, Run 2: 9.26s, Run 3: 9.24s (**Median: 9.26s**). S3 file transfer accounted for ~3.1s, while model weight deserialization and graph initialization took ~6.1s.
• **Load Test Throughput & Latency:** During the 2-minute Locust benchmark (4 concurrent users), the dual-replica deployment sustained **27.23 req/s** with a median latency of **130 ms**, a 95th percentile latency of **250 ms** (far below the 5,000 ms ceiling), and an exact **0.0% error rate** over 3,264 requests.
• **Zero-Downtime Reliability:** Rolling update from model v1 to v2 and rollback undo via `kubectl rollout undo` executed under active load without dropping a single request.

5. Proposed Improvements & What Could Have Been Done Better

1. **Token-Level Logit Distillation with Projection Heads:** While sequence-level distillation worked well, combining it with KL divergence over top-k teacher logits and intermediate hidden-state projection layers (FitNets) would help the student capture subtler slot boundary distinctions.
2. **INT8 Post-Training Quantization (PTQ) & ONNX Runtime:** Quantizing Pythia-70M from FP32 to 8-bit integers using ONNX Runtime would shrink the model footprint from ~280 MB to ~70 MB and cut CPU inference latency down to under 20 ms.
3. **Multi-Node High Availability & Disruption Budgets:** A local single-node cluster cannot protect against node failures. In production, I would deploy across multi-availability-zone worker nodes with `PodDisruptionBudgets (minAvailable: 1)` and pod anti-affinity.
4. **eBPF-Based Stateful Network Policies:** Stateless NACLs require exposing broad ephemeral port ranges for return traffic. Replacing them with Cilium eBPF policies would enable stateful Layer 7 traffic filtering without opening unnecessary port ranges.

6. Concluding Reflection

This project provided an invaluable hands-on journey through the full lifecycle of cloud-native machine learning systems. I experienced firsthand the engineering trade-offs between model parameter scale, CPU execution constraints, container memory ceilings, and automated cloud reliability. Tackling real hardware bottlenecks—from autograd matrix multiplication chokepoints to S3 volume management and stateless packet routing—has given me genuine, practical confidence in building resilient, end-to-end cloud infrastructure.

Submission Tag: final **Verified By:** Nemapu Navaneeth (Roll No: 230101086)